

DAY 1 · MODULE 3 · 30 MIN

Prompts & Context Engineering

Everything that goes into the model on every turn

Why this module

Agents don't make prompt engineering obsolete — they concentrate it.

- Instructions — the system prompt that defines behavior
- Tool descriptions the model reads to decide when to call something
- Structured output schemas whose field names and docstrings are prompts too

Sloppy prompts here cascade through your whole workflow.

Anatomy of a good system prompt

- Role and scope — who the agent is, what it may and may not help with
- How to respond — tone, length, format, when to say 'I don't know'
- How to use tools — when to call which tool, and what to do with the result
- Boundaries and safety — refusals, privacy, tenant scope

Under ~500 words. Longer often means tools or knowledge should be doing the work instead.

Anti-patterns

- The wall of rules — 40 numbered 'do not' statements. Something always gets violated.
- The example dump — ten worked examples in the system prompt. Move to few-shot messages.
- Prompting the wrong layer — 'search the docs for X' is a tool call, not a system-prompt line.
- Vague identity — 'you are a helpful assistant.' Every model already thinks that.
- Forgetting the audience — a prompt for engineers looks different than one for associates.

Two techniques worth investing in

1. Structured instructions

- Short, labeled sections beat a wall of paragraphs
- Models attend better to `## Role / ## Rules / ## Output format`
- Consistent structure helps you diff prompts across versions

2. Explicit output contracts

- Tell the model exactly what shape you want
- Better yet — use structured outputs (typed models — Day 3)
- The shape is enforced, not requested

What's actually in the context window?

Prompts are one input. On every turn the model actually sees:

- System instructions — the agent's persona, rules, output format
- The user message — what the human just typed
- Session history — prior turns, the multi-turn memory
- Tool outputs — results from any tool the model called
- Retrieved documents — chunks pulled from RAG / Foundry IQ
- Injected context — anything a context provider added (user profile, time, memory)

The system prompt is what you write once. Everything else is what you have to design. That's context engineering.

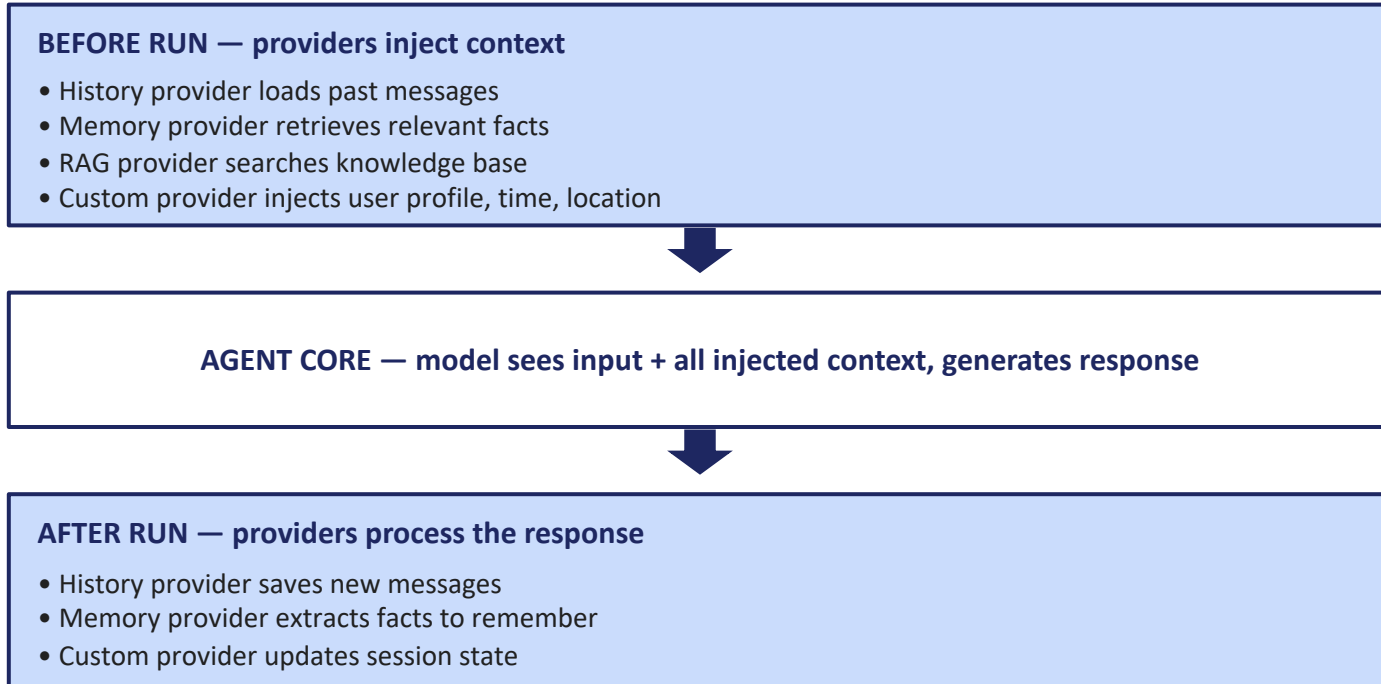
Two ways to get information into the model

Aspect	Tools	Context providers
Trigger	Reactive — model decides when to call	Proactive — runs on every invocation
Control	Model-driven (which tool, when, args)	Developer-driven (always available)
Visibility	Model must judge tool relevance	Injected as part of the prompt
Use case	On-demand actions and lookups	Always-present context
Token cost	Only when the tool is called	Every invocation

Rule of thumb: if the agent should have this info every single time, use a provider. If only when relevant, use a tool.

The context lifecycle

Every `agent.run(...)` call has three phases. Context providers hook into the first and third.



You register providers once when creating the agent. They participate in every invocation.

D E M O

~4 min

Tool vs. context provider — which fires when?

Same agent, same question, two configurations. Left pane wires `get_user_orders` as a tool — model decides to call, latency shows. Right pane injects the same data via a context provider — data was already in the prompt, answer is instant. Latency delta + reasoning delta prove the tools-vs-providers distinction.

Runbook: [demos/day1/module-3-demo-2-tool-vs-provider.md](#)

The tradeoffs

Consideration	What can go wrong
Token budget	Injected context consumes tokens every turn. Unbounded → truncation, important info lost.
Retrieval latency	Providers hitting DBs / search / APIs add latency to every call. Cache, pool, async.
Relevance	Irrelevant context doesn't just waste tokens — it degrades responses by diluting signal.
Staleness	Cached or preloaded context goes stale. Design refresh cadence deliberately.
Composability	Multiple providers writing the same window interact unexpectedly. Test them together.

Compaction (summarizing older history) is the escape valve when context grows. Day 3 memory module covers it.

Iteration loop

- Pick a small eval set (5–20 realistic inputs with expected behaviors)
- Run the current prompt against it
- Read the failures
- Change one thing
- Rerun

You'll see this loop again on Day 2 (retrieval eval) and Day 4 (workflow eval). The habit starts today.

Prompts inside MAF

```
from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient
from azure.identity import AzureCliCredential

agent = Agent(
    client=FoundryChatClient(credential=AzureCliCredential()),
    name="DocsAssistant",
    instructions=(
        "## Role\n"
        "You are a technical documentation assistant.\n\n"
        "## Rules\n"
        "- Cite sources for factual claims.\n"
        "- Say 'I don't know' when you don't.\n"
    ),
)
```

Tool descriptions are prompts too

```
@tool
def create_ticket(title: str, body: str, priority: str) -> str:
    """Create a support ticket.

    Use this when the user reports a problem that needs a human
    engineer to resolve. Do NOT use for questions you can answer
    from documentation. Priority must be one of low, med, high.
    """

    ...
```

Bad docstrings = the model calls the wrong tool at the wrong time.

Takeaways

- Prompts still matter — you'll just write fewer of them, more carefully.
- Structure > prose. Contracts > pleas.
- Iterate against an eval set, not intuition.
- Every instruction, tool docstring, and output schema is a prompt.

Next: MAF 101 — the core primitives you'll use through the rest of the workshop.